Experiment 'p_5_7_count_words' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_5_7_count_words
Natural language statement: Write a method that returns the count of all words in a string. Words are separated by spaces. For example, countWords("Mary had a little lamb") should return 5.
Method signature: p_5_7_count_words(str: string) returns (word_count: int)

Ensures

- `word_count == count_words(str)`

Functional Code Given

```
function remove_leading_spaces(s: string): (result: string)
    ensures |s| > 0 && s[0] == ' ' ==> |result| < |s|
{
    if |s| == 0 then s
    else if s[0] == ' ' then remove_leading_spaces(s[1..])
    else s
}

function remove_word(s: string): (result: string)
    ensures |s| > 0 && s[0] != ' ' ==> |result| < |s|
{
    if |s| == 0 then s
    else if s[0] != ' ' then remove_word(s[1..])
    else s
}

function count_words(s: string): int
    decreases |s|
{
```

```
    var s2 := remove_leading_spaces(s);
    if |s2| > 0 then 1 + count_words(remove_word(s2))
    else 0
}
```

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that returns the count of all words in a string. Words are separated by
    ↪ spaces. For example, countWords("Mary had a little lamb") should return 5.

The signature should be:

p_5_7_count_words(str: string) returns (word_count: int)

The method should respect the following contract:

ensures word_count == count_words(str)

The contract uses the following dafny code:

function remove_leading_spaces(s: string): (result: string)
    ensures |s| > 0 && s[0] == ' ' ==> |result| < |s|
{
    if |s| == 0 then s
    else if s[0] == ' ' then remove_leading_spaces(s[1..])
    else s
}

function remove_word(s: string): (result: string)
    ensures |s| > 0 && s[0] != ' ' ==> |result| < |s|
{
    if |s| == 0 then s
    else if s[0] != ' ' then remove_word(s[1..])
    else s
}

function count_words(s: string): int
    decreases |s|
{
    var s2 := remove_leading_spaces(s);
    if |s2| > 0 then 1 + count_words(remove_word(s2))
    else 0
}

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnSDnc5sEvKOBMudbkFB1U2Md3VdY
Model: gpt-5.5-2026-04-23
Created at: 1780678927
Finish reason: stop
Usage: 748 tokens in, and 3587 tokens out

```
{
  if |str| == 0 {
    word_count := 0;
    assert remove_leading_spaces(str) == str;
    assert count_words(str) == 0;
  } else if str[0] == ' ' {
    var rest := p_5_7_count_words(str[1..]);
    word_count := rest;
    assert str[1..] == str[1..];
    assert remove_leading_spaces(str) == remove_leading_spaces(str[1..]);
    assert count_words(str) == count_words(str[1..]);
  } else {
    var i := 0;
    while i < |str| && str[i] != ' '
      invariant 0 <= i <= |str|
      invariant remove_word(str) == remove_word(str[i..])
    {
      assert |str[i..]| > 0;
      assert str[i..][0] == str[i];
      assert str[i..][1..] == str[i + 1..];
      assert remove_word(str[i..]) == remove_word(str[i + 1..]);
      i := i + 1;
    }

    assert i > 0;
    if i < |str| {
      assert str[i] == ' ';
      assert |str[i..]| > 0;
      assert str[i..][0] == ' ';
      assert remove_word(str[i..]) == str[i..];
    } else {
      assert i == |str|;
      assert str[i..] == "";
      assert remove_word(str[i..]) == "";
    }
    assert remove_word(str) == str[i..];

    var rest := p_5_7_count_words(str[i..]);
    word_count := 1 + rest;
    assert remove_leading_spaces(str) == str;
    assert count_words(str) == 1 + count_words(remove_word(str));
    assert count_words(str) == 1 + count_words(str[i..]);
  }
```

```
}
```

Final Program

```
function remove_leading_spaces(s: string): (result: string)
    ensures |s| > 0 && s[0] == ' ' ==> |result| < |s|
{
    if |s| == 0 then s
    else if s[0] == ' ' then remove_leading_spaces(s[1..])
    else s
}

function remove_word(s: string): (result: string)
    ensures |s| > 0 && s[0] != ' ' ==> |result| < |s|
{
    if |s| == 0 then s
    else if s[0] != ' ' then remove_word(s[1..])
    else s
}

function count_words(s: string): int
    decreases |s|
{
    var s2 := remove_leading_spaces(s);
    if |s2| > 0 then 1 + count_words(remove_word(s2))
    else 0
}

method p_5_7_count_words(str: string) returns (word_count: int)
       ensures word_count == count_words(str)
{
  if |str| == 0 {
    word_count := 0;
    assert remove_leading_spaces(str) == str;
    assert count_words(str) == 0;
  } else if str[0] == ' ' {
    var rest := p_5_7_count_words(str[1..]);
    word_count := rest;
    assert str[1..] == str[1..];
    assert remove_leading_spaces(str) == remove_leading_spaces(str[1..]);
    assert count_words(str) == count_words(str[1..]);
  } else {
    var i := 0;
    while i < |str| && str[i] != ' '
      invariant 0 <= i <= |str|
      invariant remove_word(str) == remove_word(str[i..])
    {
      assert |str[i..]| > 0;
      assert str[i..][0] == str[i];
      assert str[i..][1..] == str[i + 1..];
```

```
    assert remove_word(str[i..]) == remove_word(str[i + 1..]);
    i := i + 1;
  }

  assert i > 0;
  if i < |str| {
    assert str[i] == ' ';
    assert |str[i..]| > 0;
    assert str[i..][0] == ' ';
    assert remove_word(str[i..]) == str[i..];
  } else {
    assert i == |str|;
    assert str[i..] == "";
    assert remove_word(str[i..]) == "";
  }
  assert remove_word(str) == str[i..];

  var rest := p_5_7_count_words(str[i..]);
  word_count := 1 + rest;
  assert remove_leading_spaces(str) == str;
  assert count_words(str) == 1 + count_words(remove_word(str));
  assert count_words(str) == 1 + count_words(str[i..]);
 }
}
```

Total Token Usage

Input tokens: 748
Output tokens: 3587
Reasoning tokens: 3072
Sum of 'total tokens': 4335

Experiment Timings

Overall Experiment started at 1780678925736, ended at 1780679051566, lasting 125830ms (125.83 seconds)
Iteration #1 started at 1780678925742, ended at 1780679051566, lasting 125824ms (125.82 seconds)